

Aula 09: A Jornada para a Orientação a Objetos (POO)

Esta aula é um marco no seu aprendizado. Vamos deixar de apenas "escrever scripts" para começar a "construir sistemas". Como este é um tema denso, dividiremos esta aula em vários momentos.

PARTE 1: O Conceito (O que é e por que usar?)

1.1 A Analogia do Mundo Real

Imagine que você tem uma **forma de bolo**.

- A **Forma** é a **Classe**: Ela é o projeto, o molde. Ela define o tamanho e o formato, mas você não pode comer a forma.
- O **Bolo** é o **Objeto**: É o que você cria usando a forma. Você pode ter um bolo de chocolate e um de morango. Ambos vieram da mesma forma, mas cada um tem sua cor e sabor.

No nosso sistema:

- **Classe Produto**: É o molde que diz que todo produto tem nome, preço e estoque.
- **Objeto**: É o "Teclado Mecânico" ou o "Mouse Gamer" real.

1.2 Procedural vs Orientado a Objetos

- **No Procedural (como fazíamos)**: Você tem dados (arrays) e funções soltas. É como ter as peças de um carro espalhadas pelo chão.
 - **Na POO (como faremos)**: Você une dados e funções em uma única unidade inteligente (o Objeto). É como ter o carro montado, onde você apenas gira a chave e ele sabe como ligar.
-

PARTE 2: O Molde (Criando a Classe e Atributos)

Os **Atributos** são as características do objeto (O que ele **TEM**).

```
<?php

class Produto {
    // Atributos (Estado do Objeto)
    public $id;
    public $nome;
    public $preco;
    public $quantidade;
    public $descricao;
}
```

Por que isso é melhor que um array? Diferente de um array ['nome' => '...'], na classe o PHP te ajuda. Se você tentar acessar `$produto->nomee` (errado), o VS Code vai te avisar que esse atributo não existe no molde.

PARTE 3: A Inteligência (Métodos e `$this`)

Os **Métodos** são as ações do objeto (O que ele **SABE FAZER**). Para que a classe leia seus próprios dados internos, usamos a palavra mágica `$this` (que significa "**este** objeto aqui").

```
class Produto {
    public $nome;
    public $preco;
    public $quantidade;

    // Método: Formatar o preço para a tela
    public function exibirPreco() {
        return "R$ " . number_format($this->preco, 2, ',', '.');
    }

    // Método: Gerar link de venda para WhatsApp
    public function gerarLinkWhatsApp() {
        $texto = "Olá! Tenho interesse no " . $this->nome . " que custa " . $this->exibirPreco();
        return "https://wa.me/5511999999999?text=" . urlencode($texto);
    }
}
```

Explicação Pedagógica: Quando chamamos `$this->nome`, o PHP entende: "Pegue o nome **deste** bolo específico que estou segurando agora, e não de outro".

PARTE 4: Dando Vida (Instanciação com `new`)

Para criar um objeto real a partir da classe, usamos a palavra `new`.

```
// 1. Criamos um objeto novo (Instanciamos)
$p1 = new Produto();

// 2. Preenchemos os dados
$p1->nome = "Monitor Gamer";
$p1->preco = 1200.00;

// 3. Usamos a "inteligência" dele
echo $p1->exibirPreco();
echo "<a href='" . $p1->gerarLinkWhatsApp() . "'>Comprar</a>";
```

PARTE 5: A Fábrica (Integrando com o Banco de Dados)

Agora o "pulo do gato": Como transformamos uma linha do MySQL em um Objeto Produto? Usaremos um **Método Estático** (`static`), que funciona como uma fábrica dentro da forma de bolo.

```
class Produto {
    // ... atributos anteriores ...

    /**
     * MÉTODO ESTÁTICO: Funciona como uma fábrica.
     * Ele vai ao banco e nos devolve um Objeto Produto pronto.
     */
    public static function buscarPorId($id, $pdo) {
        $stmt = $pdo->prepare("SELECT * FROM produtos WHERE id = :id");
        $stmt->execute(['id' => $id]);
        $dados = $stmt->fetch(PDO::FETCH_ASSOC);

        if (!$dados) return null;

        // A MÁGICA: Criamos o objeto e preenchemos ele aqui dentro!
        $p = new Produto();
        $p->id = $dados['id'];
        $p->nome = $dados['nome'];
        $p->preco = $dados['preco'];
        $p->quantidade = $dados['quantidade'];
        $p->descricao = $dados['descricao'];

        return $p;
    }
}
```

Uso na prática:

```
$meuProduto = Produto::buscarPorId(5, $pdo);
echo $meuProduto->exibirPreco(); // Tudo em uma linha!
```

PARTE 6: Organização e Exercícios

Onde salvar? (Padrão MVC)

Para manter a organização profissional, criaremos uma pasta chamada `models/` e salvaremos nossa classe como `models/Produto.php`.

Exercícios de Fixação

1. **Refatoração:** No método `exibirPreco()`, adicione uma lógica para que, se o preço for zero, retorne "Grátis / Brinde".
 2. **Novo Método:** Crie o método `exibirBadgeEstoque()` que retorna um `<span>` verde se houver estoque e um vermelho se estiver esgotado.
 3. **Desafio Master:** Tente criar o método estático `listarTodos($pdo)` que retorna um array contendo vários objetos do tipo `Produto`.
-

Reflexão para a Próxima Aula:

"Se eu posso alterar o preço diretamente com `$p->preco = -500;`, como posso proteger meu objeto de receber dados absurdos?" (*Isso se chama **Encapsulamento**, tema da nossa próxima semana!*)